

Vergisst nicht euer Post mortem zu schreiben und bei "Zusammenfassung: Was wurde gemacht bzw. was wurde auf dem Meeting besprochen?" hinzuschreiben was "wir" bei den Aufgaben gemacht haben :) jedenfalls für eure eigene Aufgabe, ich schreibe z.B. bei mir was zu Nr. 1 und 2 rein eheheh

Link zum AB:

<https://github.com/Compiler-CampusMinden/CB-Vorlesung-Bachelor-W25/blob/master/homework/sheet01.md>

Links zur Vorlesung, alles was man für heute braucht:

<https://github.com/Compiler-CampusMinden/CB-Vorlesung-Bachelor-W25/blob/master/lecture/01-lexing/regular1.md>

Wie man ein Post mortem schreibt ("bei Hinweise"):

<https://github.com/Compiler-CampusMinden/CB-Vorlesung-Bachelor-W25>

A1.1 Sprachen von regulären Ausdrücken

Welche Sprache wird von dem folgenden regulären Ausdruck beschrieben?

$a+a(b)^*a$

Der Regex beschreibt eine Sprache:

- die mit beliebig vielen a's anfängt
- danach ein a besitzt, (danach ein a oder mehrere a's und/oder ein b, diese Abfolge darf beliebig oft wiederholt werden)
- mit einem a endet

A1.2 Bezeichner in Programmiersprachen

Regex für die Bezeichner der Programmiersprache:

`^[a-zA-Z](_[a-zA-Z\d])+$`

Input

```
_a_aa  
aae_aasfegwegnk  
aaa_  
a__a_a  
feffasjweiofg  
a_a_a  
a_a_a  
a__a_a  
aa  
vv  
cc  
bb
```

Output

```
_a_aa  
aae_aasfegwegnk  
aaa_  
a__a_a  
feffasjweiofg  
a_a_a  
a_a_a  
a__a_a  
aa  
vv  
cc  
bb
```

- Start mit Buchstaben
- danach 1 oder mehr Zeichen (Buchstaben, Zahlen und Unterstrich), damit der Variablenname mind. 2 Zeichen groß ist
- ein Buchstabe (kein Unterstrich)

zwei Beispiele:

pTest

- start mit p: $^{\wedge}(V|v|p|P)$
- danach beliebig viele Buchstaben, Zahlen und Unterstriche "Test": $[a-zA-Z0-9_]+$
- der Name endet nicht mit einem Unterstrich: $(?!_ \$)$
 - somit ist der Bezeichner gültig.

V_

- Start mit V, passt
- danach beliebig viele Buchstaben, Zahlen und Unterstriche (und mindestens einer): passt
- Ende mit einem Unterstrich: der Bezeichner ist ungültig.

DFAs

Tupel des DFA der Programmiersprache:

(Zustände, Alphabet, Übergangsfunktion, Startzustand, Endzustände)

Zustände:

q0: ein Startzeichen $[V|v|p|P]$ (Startzustand)

q1: nach dem Startzeichen ein Zeichen aus unserem Alphabet

unterstrichTest: bei Eingabe eines Unterstrichs

accept: unsere Eingabe ist gültig (Endzustand)

reject: eingabe ist ungültig

Alphabet von Eingabesymbolen:

0-9, a-z, A-Z, _

Übergangsfunktionen zu Zuständen:

Zustand	Eingabe	nächster Zustand
q0	V,P,v,p	q1
q0	sonst	reject
q1	a-z, A-Z, 0-9	accept
q1	_	reject
unterstrichTest	a-z, A-Z, 0-9	q2
unterstrichTest	_	reject
accept	a-z, A-Z, 0-9	accept
accept	_	unterstrichTest

Startzustand:

- q0

Endzustände (akzeptierende Zustände):

- accept

2 Beispiele:

pTest

q0 -> p -> q1
 q1 -> T -> accept
 accept -> e -> accept
 accept -> s -> accept
 accept -> t -> accept

accept ist der akzeptierende Zustand: Eingabe ist gültig

pNein_

q0 -> p -> q1
 q1 -> N -> accept
 accept -> e -> accept
 accept -> i -> accept
 accept -> n -> accept
 accept -> _ -> reject

reject ist kein Akzeptierender Zustand: Eingabe ist ungültig

reguläre Grammatik

versuch:

$G = (N, T, P, S)$

Nichtterminale = (S, A, B) (Startsymbol, A für ein weiteres Zeichen, B Zustand wird nur erreicht, falls A ein Unterstrich ist)

Terminale (alle Zeichen) = a-z, A-Z, 0-9, _

Produktionen:

$S \rightarrow vA \mid VA \mid pA \mid PA$

$A \rightarrow a A \mid \dots \mid z A, \mid A A \mid \dots \mid Z A, \mid 0 A \mid \dots \mid 9 A \mid _ B$ (alle Kleinbuchstaben, Großbuchstaben, Zahlen führen zu A, Unterstrich führt zu B)

$B \rightarrow a A \mid \dots \mid z A, \mid A A \mid \dots \mid Z A, \mid 0 A \mid \dots \mid 9 A$ (nach einem Unterstrich muss ein Zeichen folgen, dass kein Unterstrich ist)

richtig:

$S \rightarrow a A \mid b A \mid \dots \mid z A \mid A A \mid B A \mid \dots \mid Z A$

$A \rightarrow a A \mid b A \mid \dots \mid z A \mid A A \mid B A \mid \dots \mid Z A \mid _ B \mid \epsilon$

$B \rightarrow a A \mid b A \mid \dots \mid z A \mid A A \mid B A \mid \dots \mid Z A \mid _ B$